

华中科技大学

2025

系统能力培养 课程实验报告

题 目: riscv32 指令模拟器

专 业: 计算机科学与技术

班 级: CS2208 班

学 号: U202215614

姓 名: 王天睿

电 话: 13755399880

邮 件: 1772165413@qq.com

完成日期: 2026-01-14



目 录

1	课程实验概述	1
1.1	课设目的	1
1.2	课设任务	1
1.3	课设环境	2
2	实验方案设计	3
2.1	PA0 实验设计	3
2.2	PA1 实验设计	4
2.3	PA2 实验设计	5
2.4	PA3 实验设计	8
3	实验结果与结果分析.....	10
3.1	PA0 测试结果与分析	10
3.2	PA1 测试结果与分析	11
3.3	PA2 测试结果与分析	12
3.4	PA3 测试结果与分析	14
4	总结与课程建议	15
	参考文献	16

1 课程实验概述

1.1 课设目的

在代码框架中实现一个简化的 riscv32 模拟器：

1. 可解释执行 riscv32 执行代码
2. 支持输入输出设备
3. 支持异常流处理
4. 支持精简操作系统——支持文件系统
5. 支持虚存管理
6. 支持进程分时调度

最终在模拟器上运行“仙剑奇侠传”，让学生探究“程序在计算机上运行”的机理，掌握计算机软硬协同的机制，进一步加深对计算机分层系统栈的理解，梳理大学 3 年所学的全部理论知识，提升学生计算机系统能力。

1.2 课设任务

PA0 - 世界诞生的前夜：开发环境配置

PA1 - 开天辟地的篇章：最简单的计算机

PA1.1 简易调试器 PA1.2 表达式求值 PA1.3 监视点与断点

PA2 - 简单复杂的机器：冯诺依曼计算机系统

PA2.1 运行第一个 C 程序 PA2.2 丰富指令集，测试所有程序 PA2.3 实现 I/O 指令，测试打字游戏

PA3 - 穿越时空的旅程：批处理系统

PA3.1 实现系统调用 PA3.2 实现文件系统 PA3.3 运行仙剑奇侠传

1.3 课设环境

Aliyun 阿里云 弹性云服务器 ECS 2 核 2GiB

存储空间：40GiB

操作系统版本：ubuntu 20.04

其他工具：vim, Git, GDB

实例信息			
健康状态	付费类型		
正常 健康诊断	按量 按量付费 转包年包月 🔗		
自动释放时间	实例规格		
- 释放	ecs.e-c1m1.large 🔗 🔗 更换 购买相同配置		
CPU & 内存	公网 IP		
2 核 (vCPU) 2 GiB 更改 CPU 选项 调整配置	47.115.224.194 🔗 转换为弹性公网 IP		
主私网 IP	操作系统 🔗		
172.29.104.61 🔗	Ubuntu 20.04 64位 更换		
公网带宽	带宽计费方式		
5 Mbps (峰值) 更改带宽	按使用流量		
创建时间	系统盘		
2026年1月14日 16:24:00	d-f8zc8bdlj7w5v6zq7dc5 🔗 ESSD Entry 云盘 40 GiB		

2 实验方案设计

2.1 PA0 实验设计

PA0 git commit 的带签名的 git log 记录如下：

```
ecs-user@iZf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1/nemu$ git add .
ecs-user@iZf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1/nemu$ git commit --allow-empty
[pa0 2d43abe] finish pa0
ecs-user@iZf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1/nemu$ git log
commit 2d43abea06bb21ae34adf9d85b0408d5d8458460 (HEAD -> pa0)
Author: Wang TianruiU202215614 <1772165413@qq.com>
Date:   Wed Jan 14 17:41:48 2026 +0800

    finish pa0
```

图 2.1 PA0 git log

PA0 主要是配置实验环境，没有太多的设计思路可言。仅展示一些配置环境过程中遇到的问题或要点。

阅读手册，注意下载以下工具和 vim：

Installing tools for PAs

The following tools are necessary for PAs:

```
apt-get install build-essential # build-essential packages, include binary utilities, gcc, make
apt-get install man             # on-line reference manual
apt-get install gdb             # GNU debugger
apt-get install git             # reversion control system
apt-get install libreadline-dev # a library to use compile the project later
apt-get install libsdl2-dev     # a library to use compile the project later
apt-get install libc6-dev-i386  # a library to use compile the project later
apt-get install qemu-system     # QEMU
```

The usage of these tools is explained later.

图 2.2 工具下载

如果没有下载上面的工具，会出现类似下面的报错，例如 make 时候发现没装 SDL2 的开发包

```
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1/nemu$ make
Building x86-nemu
+ CC src/memory/memory.c
+ CC src/device/device.c
+ CC src/device/vga.c
+ CC src/device/keyboard.c
src/device/keyboard.c:3:10: fatal error: SDL2/SDL.h: No such file or directory
  3 | #include <SDL2/SDL.h>
    |          ^~~~~~
compilation terminated.
make: *** [Makefile:50: build/obj-x86/device/keyboard.o] Error 1
```

图 2.3 环境缺失

可能会出现 qemu-diff 缺失的问题，可以修改 Makefile 将其注释掉使得程序正常运行。

2.2 PA1 实验设计

PA1 提交记录如下：

```
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1$ make submit
STUID=U202215614 STUNAME=王天睿 bash -c "$(curl -s https://course1.istratus.cn/pa/scripts1/submit.sh)"
The task to be submitted >
1) PA1
2) PA2
3) PA3
4) PA4
5) PA5
6) Report
#? 1
Input your submission remark:
finish pa1
==Pushing current branch: pa1 to remote repo...
Username for 'https://course1.istratus.cn': U202215614
Password for 'https://U202215614@course1.istratus.cn':
Enumerating objects: 1684, done.
Counting objects: 100% (1684/1684), done.
Delta compression using up to 2 threads
Compressing objects: 100% (1357/1357), done.
Writing objects: 100% (1684/1684), 2.85 MiB | 2.96 MiB/s, done.
Total 1684 (delta 300), reused 1609 (delta 258)
remote: Resolving deltas: 100% (300/300), done.
To https://course1.istratus.cn/git/U202215614.git
 * [new branch]      pa1 -> pa1
==Submitting PA1...
{"status":"ok","data":true}
==Done!
```

图 2.4 PA1 提交

PA1 最终要求实现一个简易调试器，相当于一个简化版的 gdb，要求能实现单步执行、打印寄存器状态、扫描内存、表达式求值、监视点等功能。

阶段 1：实现“单步、打印寄存器状态、扫描内存”三个调试功能

阶段 2：实现调试功能的表达式求值

阶段 3：实现监视点

PA1.1 首先需要实现单步执行，打印寄存器状态，扫描内存这三个函数。这三个函数在 ui.c 中定义为 cmd_si，cmd_info 和 cmd_x。cmd_si 调用 cpu_exec 执行指定的步数，cmd_info 调用 isa_reg_display 进行寄存器名称与值的输出，isa_reg_display 中使用函数 reg_name 和宏 reg_1 进行寄存器名称与值的打印，

`cmd_x` 中对输入的参数进行分割, 使用 `vaddr_read` 函数读出对应地址的值然后打印出来。

PA1.2 进行表达式求值, 首先要在 `expr.c` 中补充规则和相应的正则表达式, 例如十六进制数的正则表达式与 `token` 类型为 `{"0[Xx][0-9a-fA-F]+", TK_HEX}`。随后完成函数 `make_token`, 用于为算术表达式中的各种 `token` 类型添加规则, 并在成功识别出 `token` 后, 将 `token` 的信息依次记录到 `tokens` 数组中。接着完成括号匹配函数, 完成运算符优先级函数, 根据 C 语言中的优先级来实现对应的操作符的优先级。在这些辅助函数都实现后, 就可以进行递归求值函数 `eval` 的编写, `eval` 对传入的参数 `p`、`q` 进行处理, 如果 `p>q` 则直接返回 0, `success` 置为 `false`; 如果 `p==q` 则判断该 `token` 的类型是不是整数、十六进制整数和寄存器, 如果是则返回其值, 如果不是则置 `success` 为 `false`, 返回 0; 随后进行括号匹配检查, 如果匹配则返回 `eval(p+1,q-1)`, 否则则进入寻找主操作符行为; 如果能找到主操作符, 则根据主操作符返回相应的值, 如果找不到则置 `success` 为 `false`, 返回 0。完成 `eval` 函数后, 将 `eval` 函数和 `make_token` 函数整合到函数 `expr` 中, 即可完成整个表达式求值框架的编写。

PA1.3 要求实现监视点功能, 首先要完善 `watchpoint` 结构, 补充相应的变量, 因为要监测表达式, 所以需要一个 `char` 数组存放表达式, 同时因为要监测值的变化, 所以需要一个变量用于存储旧值。随后在 `watchpoint.c` 中实现函数 `new_wp`, `free_wp`, `print_wp`。`new_wp` 用于新建一个监视点, 把链表 `free_` 中的节点放入链表 `head` 中; `free_wp` 用于释放监视点, 根据编号在 `head` 链表找到相应的节点, 放入链表 `free_`, 这两个函数的操作都是通过链表的插入与删除来实现的, 且为了方便快捷在插入链表时都选择头插法。`print_wp` 函数用于打印监视点信息, 不再赘述。完成上述函数后, 在 `ui.c` 中使用上述函数完成命令 `cmd_x` 与 `cmd_d`, 完善函数 `cmd_info`, 同时要在 `cpu.exec` 中实现当监视点状态改变使程序暂停执行的逻辑, 该逻辑较为简单, 程序每执行一步就对所有的监视点进行表达式求值, 并与旧值进行比较, 如果两值不同, 则置 `nemu` 的状态为 `NEMU_STOP`。

2.3 PA2 实验设计

PA2 提交记录如下:

```

ecs-user@izf8zc8bd1j7w5v6zvmu61z:~/ics2019_1$ git checkout pa2
Switched to branch 'pa2'
ecs-user@izf8zc8bd1j7w5v6zvmu61z:~/ics2019_1$ make submit
STUID=U202215614 STUNAME=王天睿 bash -c "${curl -s https://course1.istratus.cn/pa/scripts1/submit.sh}"
The task to be submitted >
1) PA1
2) PA2
3) PA3
4) PA4
5) PA5
6) Report
#? 2
Input your submission remark:
finish pa2
==Pushing current branch: pa2 to remote repo...
Username for 'https://course1.istratus.cn': U202215614
Password for 'https://U202215614@course1.istratus.cn':
Enumerating objects: 116, done.
Counting objects: 100% (115/115), done.
Delta compression using up to 2 threads
Compressing objects: 100% (66/66), done.
Writing objects: 100% (69/69), 13.08 KiB | 2.18 MiB/s, done.
Total 69 (delta 43), reused 0 (delta 0)
To https://course1.istratus.cn/git/U202215614.git
 * [new branch]      pa2 -> pa2
==Submitting PA2...
{"status":"ok","data":true}
==Done!

```

图 2.5 PA2 提交

PA2 要求实现足够多的指令，能够运行各种测试，同时提供对输入输出设备的支持。

阶段一：要求实现部分指令能够运行 `dummy.c`，观察反汇编代码，找出未实现的指令，编写相应的译码和执行函数即可。

阶段二：要求实现更多的指令，在 NEMU 中运行所有 `cpptest`，这里需要找出观察所有的测试文件的反汇编代码，找出未实现的指令，编写相应的译码和执行函数。

阶段三：要求能够运行打字小游戏以及其他的输入输出测试，这个需要在相关的设备文件下编写设备功能，同时完成相关输入输出函数。

PA2.1 首先直接使用 `make` 命令跑 `dummy` 程序，会显示 `abort`，随后会在 `build` 文件夹中生成对应的反汇编文件，将其中未实现的指令找出，在手册中查询，找出所有的伪指令，最终可确认 `dummy` 需要实现的新指令有 `auipc addi jal jalr`，随后在 `all-instr.h` 中定义相关指令的执行函数，在 `exec.c` 的 `opcode_table` 中添加新指令，在 `decode.c` 中实现相应的译码辅助函数，在 `rtl.h` 中修改完善 `rtl` 指令，在 `compute.c` 和 `control.c` 等文件中使用 `rtl` 指令实现正确的执行辅助函数，完成上述步骤后，重新编译运行即可。

PA2.2 过程类似于 PA2.1，只不过要实现更多的指令。更多的指令意味着更容易出错，因此首先完成 `diff-test` 是一个不错的选择，`diff-test` 将自己实现的 `nemu` 与 `qemu` 在执行指令的过程中进行对比，每执行一步就比较两者 32 个寄存器中的值是否一样，如果不一样则报错，根据 `abort` 的 PC 值可以在反汇编代码中找

出是哪一行出错，从而分析找出解决方法。完成 diff-test 后，可以开始指令的实现，为了方便测试找出错误，我选择将测试文件按大小从小到大排列，依次编译运行，找出未实现的指令进行实现。这里以 sum.c 为例，阐述实现指令的整个流程：在完成了 dummy，观察 sum 的反汇编代码，发现还需要实现的指令有 beq、bne、add 和 sltiu，其中 beq 和 bne 指令是 B 型指令，add 是 R 型指令，在 dummy 中没涉及到这两个类型的指令，所以要进行译码辅助函数的编写，查阅手册根据相关指令的结构编写好对应的译码函数，随后进行执行辅助函数的编写，beq 指令和 bne 指令因为设计指令的跳转，执行辅助函数要在 control.c 中编写，add 和 sltiu 是单纯的计算指令，所以在 compute.c 文件中编写，随后查阅手册，根据指令的相关行为编写对应的执行辅助函数。编写完两个函数后，如果没有声明，需要在 decode.h 中声明译码辅助函数，需要在 all-instr.h 中声明执行辅助函数。随后在 exec.c 的 opcode_table 中指明指令对应的译码和执行辅助函数，做完这些后，使用 make 命令执行对应的测试文件，如果出错就检查对应的译码和执行函数，通过就进入下一个指令的编写，重复上述过程直到所有测试文件通过。此外，在测试文件 hello-str 和 string 中要使用库函数 sprintf、strcmp、strcat、strcpy 和 memset，这些函数需要我们在 nexus-am/libs/klib/src/stdio.c 以及 string.c 中编写，string.c 中需要编写的是平常经常使用到的字符串相关的函数，较为简单例如 strcmp、strlen、strcat 等，在 stdio.c 中需要编写 sprintf、printf 等函数，这两个函数都通过中间函数 vsprintf 实现。vsprintf 需要对传入的字符串 fmt 以及可变参数列表 va_list 进行处理，当在 fmt 字符串中读取到%d、%x、%s 的子串时，调用 va_arg 函数将 va_list 中对应类型的参数，然后转换成字符串传入输出字符串 out 中，这样就完成了 vsprintf 函数的编写。在 sprintf 和 printf 函数中简单的调用 vsprintf 函数即可完成各自的功能。至此 PA2.2 的内容全部完成，使用一键回归测试可以测试所有的程序。

PA2.3 要完成串口、时钟、键盘、VGA 四个输入输出设备程序的编写。串口在 trm.c 中已经实现，我们还需要编写 printf 函数以便程序运行，由于 printf 函数已在 PA2.2 中写好，不再赘述。时钟的功能需要在 nemu-timer.c 中完善，在启动 init 时通过 RTC_ADDR 获取启动时间，运行时钟功能时通过地址 RTC_ADDR 获取当前时间，将 uptime->hi 设为 0，uptime->lo 设为当前时间与

启动时间的差值，即可完成时钟功能。键盘的功能需要在 `nemu-input.c` 中完善，通过地址 `KBD_ADDR` 获取键盘按键信息，放入 `kbd->keycode` 中，将按键信息与 `KEYDOWN_MASK` 相与，如果值为 1 就说明是按下，值为 0 说明是松开。最后 需要实现的是 VGA 设备，在 `nemu-video.c` 中完善相关的功能，VGA 设备需要将 `pixels` 中的像素信息写入 VGA 对应的地址空间中，就能正确显示图像。

2.4 PA3 实验设计

PA3 提交记录如下：

```
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1$ git checkout pa3
Switched to branch 'pa3'
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1$ make submit
STUID=U202215614 STUNAME=王天睿 bash -c "$(curl -s https://course1.istratus.cn/pa/scripts1/submit.sh)"
The task to be submitted >
1) PA1
2) PA2
3) PA3
4) PA4
5) PA5
6) Report
#? 3
Input your submission remark:
finish pa3
==Pushing current branch: pa3 to remote repo...
Username for 'https://course1.istratus.cn': U202215614
Password for 'https://U202215614@course1.istratus.cn':
Enumerating objects: 121, done.
Counting objects: 100% (121/121), done.
Delta compression using up to 2 threads
Compressing objects: 100% (57/57), done.
Writing objects: 100% (61/61), 13.45 KiB | 1.68 MiB/s, done.
Total 61 (delta 33), reused 1 (delta 0)
To https://course1.istratus.cn/git/U202215614.git
* [new branch]      pa3 -> pa3
==Submitting PA3...
{"status":"ok","data":true}
==Done!
```

图 2.6 PA3 提交

PA3 的主要任务是实现系统调用和文件系统，使得 `nemu` 最终能够运行仙剑奇侠传。

一阶段：要求实现自陷操作 `_yield` 及其过程，要实现自陷操作，首先要在 `cpu` 结构中添加控制状态寄存器，然后实现自陷需要的指令，通过异常号识别出自陷异常，完成自陷事件。

二阶段：要实现用户程序的加载和系统调用，支撑 TRM 程序的运行，需要通过实现 `loader` 函数，增加系统调用，完善堆区管理函数。

三阶段：需要实现文件系统，完成 `fs_open`, `fs_read` 等函数，完成设备函数的编写，使用 `fs` 函数修改 `loader`，最终可运行仙剑奇侠传。

PA3.1 要实现自陷操作，要完成自陷操作，首先要实现自陷指令，需要实现的指令有 `csrrs`, `csrrw`, `ecall` 和 `sret`，通过 `ecall` 指令进入自陷操作，`csrrs` 和 `csrrw` 指令对控制状态寄存器进行修改，`sret` 指令用于自陷后返回，然后需要在 `intr.c`

文件中进行 `raise_intr` 函数的编写, `raise_intr` 函数用于模拟相应过程: 将当前 PC 值保存到 `sepc` 寄存器, 在 `scause` 寄存器中设置异常号, 从 `stvec` 寄存器中取出异常入口地址, 跳转到异常入口地址; 触发自陷操作后, 需要保存上下文, 根据 `trap.S` 汇编代码的压栈的顺序重构 `_Context` 成员体结构, 接着要实现正确的事件分发, 需要在 `__am_irq_handle` 函数中通过异常号识别出自陷异常, 在 `do_event` 函数中 识别出自陷事件 `_EVENT_YIELD`, 最后通过 `sret` 指令返回并恢复上下文。

PA3.2 需要实现用户程序的加载和系统调用, 支撑 TRM 程序的运行, 为了加载用户程序首先要实现 `loader` 函数, 因为目前还没有实现文件系统, 所以直接在 `loader` 函数中通过 `ramdisk_read` 函数把可执行文件中的代码和数据放置在正确的 内存位置, 然后跳转到程序入口, 接着需要完成系统调用的实现, 和 3.1 中识别 出自陷事件类似, 让 `nemu` 能够识别出系统调用, 然后在 `do_event` 中添加 `do_syscall` 的调用, 根据 `nanos.c` 中的 `ARGS_ARRAY` 在 `riscv32-nemu` 中实现正确 的 GPR 宏, 随后添加 `SYS_yield`、`SYS_read`、`SYS_write` 和 `SYS_brk` 系统调用, 完成函数 `_sbrk` 实现堆区管理。

PA3.3 需要实现文件系统, 添加设备的支持, 最终能够运行仙剑奇侠传。首先需要实现函数 `fs_open`, `fs_read` 和 `fs_close`, 因为 `ramdisk` 中的文件数量增加之 后, 就不适合直接在 `loader` 函数中直接使用 `ramdisk_read`, 在完成了这几个 `fs` 函 数后, 替换掉 `loader` 函数中的 `ramdisk_read` 函数, 修改其逻辑这样就可以在 14 `loader` 中使用文件名来指定加载的程序了, 随后完成 `fs_write` 和 `fs_lseek` 函数, 使其能够进行输出, 完成这些函数后, 要补充相关的系统调用; 要实现虚拟文件 系统 VFS, 把 IOE 抽象成文件, 首先需要在 VFS 中补充多种特殊文件的支持, 接着实现函数 `serial_write`, 完成串口的写入, 然后完成实现 `events_read` 函数, 支持读操作, 最后需要完成 `init_fs`、`fb_write`、`fbsync_write`、`init_device`、`dispinfo_read` 函数, 实现对 VGA 设备的支持。要注意的是因为在 `Finfo` 结构中 添加了读函数 指针和写函数指针, 所以要修改修改 `fs_write` 和 `fs_read` 的逻辑。

3 实验结果与结果分析

备注：由于 aliyun 的环境和 ics-pa-gitbook 手册上提供的有所差异，因此进行了许多其他环境的修改，例如全局环境变量、软链接和一些路径，以及 riscv 交叉编译环境的准备与手册实际有所不同。因此，如果要重现以下实验结果，请前往 aliyun 平台

3.1 PA0 测试结果与分析

Git 与 Vim 教学完成结果：

用 vim 修改 makefile 文件，修改成自己的姓名和学号，而后用 git 工具进行比对

```
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1$ vim nemu/Makefile.git
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1$ git status
On branch pa0
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   nemu/Makefile.git

no changes added to commit (use "git add" and/or "git commit -a")
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1$ git diff
diff --git a/nemu/Makefile.git b/nemu/Makefile.git
index 8e8c338..05d0a44 100644
--- a/nemu/Makefile.git
+++ b/nemu/Makefile.git
@@ -1,5 +1,5 @@
-STUID = 181220000
-STUNAME = 张三
+STUID = U202215614
+STUNAME = 王天睿

# DO NOT modify the following code!!!
```

图 3.1 Git 与 Vim 教学完成结果

Make 结果：成功 Make

```
+ CC src/isa/x86/decode/decode.c
+ CC src/isa/x86/reg.c
+ CC src/isa/x86/exec/special.c
+ CC src/isa/x86/exec/data-mov.c
+ CC src/isa/x86/exec/control.c
+ CC src/isa/x86/exec/exec.c
+ CC src/isa/x86/exec/arith.c
+ CC src/isa/x86/exec/prefix.c
+ CC src/isa/x86/exec/logic.c
+ CC src/isa/x86/exec/system.c
+ CC src/isa/x86/exec/cc.c
+ CC src/isa/x86/diff-test.c
+ CC src/isa/x86/intr.c
+ CC src/isa/x86/logo.c
+ LD build/x86-nemu
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1/nemu$
```

图 3.2 make 成功

Make run 结果：报错与手册预期一致。手册声明 PA0 中忽视此条报错。

```
ecs-user@izf8zc8bd1j7w5v6zvmu6iZ:~/ics2019_1/nemu$ make run
Building x86-nemu
./build/x86-nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,36,load_img] No image is given. Use the default build-in image.
x86-nemu: src/isa/x86/reg.c:19: reg_test: Assertion `reg_w(i) == (sample[i] & 0xffff)' failed.
make: *** [Makefile:77: run] Aborted
```

图 3.3 make run 成功

3.2 PA1 测试结果与分析

Help 指令测试：结果满足任务要求

```
./build/riscv32-nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,36,load_img] No image is given. Use the default build-in image.
[src/memory/memory.c,16,register_pmem] Add 'pmem' at [0x80000000, 0x87ffffff]
[src/monitor/monitor.c,20,welcome] Debug: ON
[src/monitor/monitor.c,21,welcome] If debug mode is on, A log file will be generated to record necessary, you can turn it off in include/common.h.
[src/monitor/monitor.c,28,welcome] Build time: 19:02:22, Jan 14 2026
welcome to riscv32-NEMU!
For help, type "help"
(nemu) help
help - Display informations about all supported commands
c - Continue the execution of the program
q - Exit NEMU
si - Single step
info - Show info of regs/watchpoint
p - Evaluate given expression
x - Scan memory
w - Set watchpoint
d - Remove watchpoint
(nemu)
```

图 3.4 help 指令测试

单步执行Si：包含参数缺省值以及指定步数的情况

```
(nemu) si
80100000: b7 02 00 80          lui  0x80000,t0
(nemu) si 2
80100004: 23 a0 02 00          sw   0(t0),$0
80100008: 03 a5 02 00          lw   0(t0),a0
```

图 3.5 Si 指令测试

打印寄存器信息：info r

```
(nemu) info r
[src/monitor/debug/ui.c,120,cmd_info] 进入info
[src/monitor/debug/ui.c,130,cmd_info] 打印参数点
$0 = 0x00000000 , ra = 0x00000000 , sp = 0x00000000 , gp = 0x00000000 , tp = 0x00000000 , t0 = 0x80000000 , t1 = 0x00000000 , t2 = 0x00000000
s0 = 0x00000000 , s1 = 0x00000000 , a0 = 0x00000000 , a1 = 0x00000000 , a2 = 0x00000000 , a3 = 0x00000000 , a4 = 0x00000000 , a5 = 0x00000000
a6 = 0x00000000 , a7 = 0x00000000 , s2 = 0x00000000 , s3 = 0x00000000 , s4 = 0x00000000 , s5 = 0x00000000 , s6 = 0x00000000 , s7 = 0x00000000
s8 = 0x00000000 , s9 = 0x00000000 , s10 = 0x00000000 , s11 = 0x00000000 , t3 = 0x00000000 , t4 = 0x00000000 , t5 = 0x00000000 , t6 = 0x00000000
```

图 3.6 打印测试

表达式求值：测试结果均正确

```
(nemu) p 1+2+3
结果为6
(nemu) p 2*5+4*7
结果为38
```

图 3.7 表达式求值测试

监视点测试：设置四个监视点后，使用 info 打印监视点信息，再删除 3 号监视点后打印监视点信息

```

(nemu) w $t0
[src/monitor/debug/watchpoint.c,26,new_wp] 新建的监视点的value为$t0
(nemu) w $t1
[src/monitor/debug/watchpoint.c,26,new_wp] 新建的监视点的value为$t1
(nemu) w $t2
[src/monitor/debug/watchpoint.c,26,new_wp] 新建的监视点的value为$t2
(nemu) w $t3
[src/monitor/debug/watchpoint.c,26,new_wp] 新建的监视点的value为$t3
(nemu) info w
[src/monitor/debug/ui.c,120,cmd_info] 进入info
WP NO.3 "$t3" value=0
WP NO.2 "$t2" value=0
WP NO.1 "$t1" value=0
WP NO.0 "$t0" value=-2147483648
(nemu) d 3
(nemu) info w
[src/monitor/debug/ui.c,120,cmd_info] 进入info
WP NO.2 "$t2" value=0
WP NO.1 "$t1" value=0
WP NO.0 "$t0" value=-2147483648

```

图 3.8 监视点测试

3.3 PA2 测试结果与分析

本部分的测试结果需要图形化，启动 aliyun 平台的 RDP（远程桌面）连接。启动后可以看到 ubuntu 操作系统界面，可以在图形化界面打开终端。



图 3.9 启动 RDP

一键回归测试:

```

[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
[ string] PASS!
[ sub-longlong] PASS!
[ sum] PASS!
[ switch] PASS!
[ to-lower-case] PASS!
[ unalign] PASS!
[ wanshu] PASS!

```

图 3.10 一键回归测试

Micro bench: 由于是 2vCPU 和 2GiB，得分会偏低。


```

Empty mainargs. Use "ref" by default
===== Running MicroBench [input *ref*] =====
[qsrt] Quick sort: * Passed.
      min time: 5516 ms [92]
[queen] Queen placement: * Passed.
      min time: 5324 ms [88]
[bf] Brainf**k interpreter: * Passed.
      min time: 39743 ms [59]
[flb] Fibonacci number: * Passed.
      min time: 428913 ms [6]
[steve] Eratosthenes sieve: * Passed.
      min time: 130769 ms [30]
[15pz] A* 15-puzzle search: * Passed.
      min time: 28424 ms [21]
[dinic] Dinic's maxflow algorithm: * Passed.
      min time: 10910 ms [99]
[lzip] Lzip compression: * Passed.
      min time: 21878 ms [34]
[ssort] Suffix sort: * Passed.
      min time: 5681 ms [79]
[md5] MD5 digest: * Passed.
      min time: 65710 ms [26]
=====
MicroBench PASS      53 Marks
                    vs. 100000 Marks (i7-7700K @ 4.20GHz)
Total time: 775686 ms
menu: HIT GOOD TRAP at pc = 0x801072a0

```

图 3.11 Micro bench 测试

slider 测试

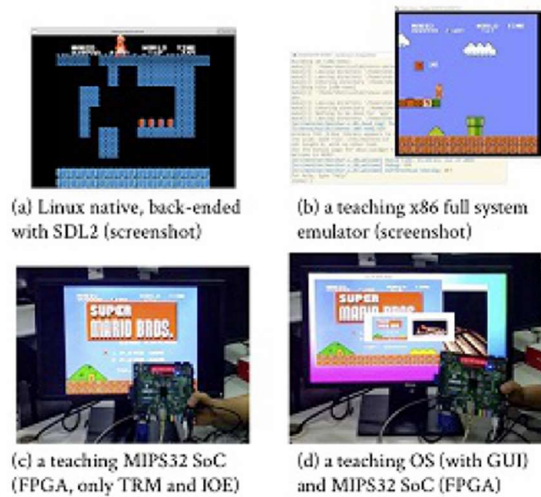


Figure 4. The same LiteNES emulator running on different platforms.

图 3.12 slider 测试

打字游戏

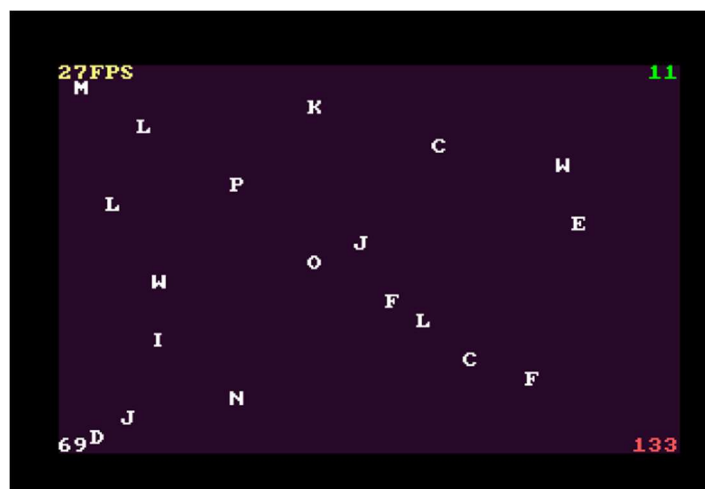
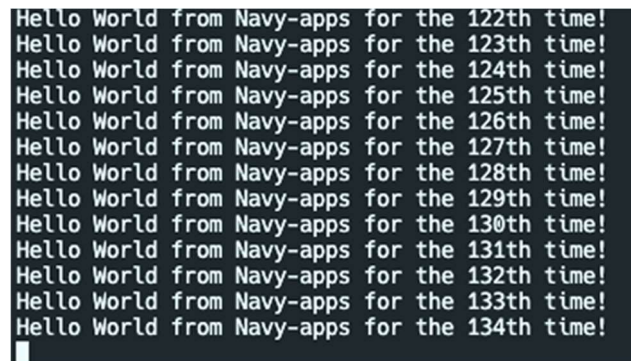


图 3.13 打字游戏测试

3.4 PA3 测试结果与分析

完成情况为 PA3.1 和 PA3.2，PA3.3 有 bug 运行无法正常。不排除是实验环境与官方不同导致的。

Hello 测试：与预期结果一致，为 PA3.1 和 3.2 完成标志。



```
Hello World from Navy-apps for the 122th time!  
Hello World from Navy-apps for the 123th time!  
Hello World from Navy-apps for the 124th time!  
Hello World from Navy-apps for the 125th time!  
Hello World from Navy-apps for the 126th time!  
Hello World from Navy-apps for the 127th time!  
Hello World from Navy-apps for the 128th time!  
Hello World from Navy-apps for the 129th time!  
Hello World from Navy-apps for the 130th time!  
Hello World from Navy-apps for the 131th time!  
Hello World from Navy-apps for the 132th time!  
Hello World from Navy-apps for the 133th time!  
Hello World from Navy-apps for the 134th time!
```

图 3.14 hello 测试

4 总结与课程建议

这次的项目可以说是整个大学期间事件跨度最长的了。同时也涉及到了非常多的知识，从最基础的 c 语言，到数据结构算法，具体的指令集，以及 vga 等更加具体的知识。在整个实现的过程中，尤其是 pa2，深刻感受到了 riscv 的简明。整个项目分为几个阶段，由于时间原因，我最终只完成到 pa3。

对于 makefile 的理解。我之前对于 makefile 一直是一知半解，对于具体的规则都不是很了解，看到 pa 中那么复杂的 makefile 也不想多看，但是在后来的必答题中发现需要回答相关的问题，就耐着性子看完 makefile 的一些基础知识，这时候再回过头看项目中涉及到的 makefile 又觉得可以理解了。

再比如项目中对于宏的频繁使用，虽然在理解的过程中确实带来了很多的麻烦，但是也让我体会到了宏的一个方面吧。当然最重要的还是对于一个“系统”运行起来的各种细节。从硬件到软件，从指令集的实现到上层简单操作系统的实现，都让我学到了很多。

对课程的建议：整体工作量有些过大了，希望减小一些；课程 PPT 有点过于单薄了，基本没有用处；官方手册仓库有些工具和链接已经过期或者隐藏了，希望及时更新。

签名处：

王天睿

参考文献

课程主页: <https://course1.istratus.cn/projects/pa/wiki>

官方手册: <https://nju-projectn.github.io/ics-pa-gitbook/ics2019/>